



Observability

Logs, metrics and traces — the concepts, before the lab

Three days. One application. Three ways of seeing it.



Day 1

Logs



Day 2

Metrics



Day 3

Traces

“Ordering fails. Sometimes.”

That is the entire bug report. It is also a real one.

? What you actually have

- Three services, two of them written by someone who left
- A customer who cannot reproduce it
- docker logs, and a terminal
- A suspicion, which is not evidence

✓ What you need to be able to answer

- How often is “sometimes”? Once an hour, or 3% of orders?
- Which service returns the failure, and which one causes it?
- Did it start at 14:20? What else happened at 14:20?
- Is it slow, or is it broken? Those are different pages.

```
$ docker logs gateway --tail 5
POST /orders 500 1243ms
POST /orders 200 89ms
POST /orders 500 2104ms
# ok. now find the other 40,000 requests, across 3 containers, from 14:00 to 14:30.
```

! Observability is not a tool you install. It is the property of being able to answer a question you did not think of in advance.

Monitoring answers old questions. Observability lets you ask new ones.

Monitoring

Questions you wrote down in advance

- You decide, at build time, what to watch
- Dashboards and alerts are fixed in place
- Great for: is the disk full, is the process up
- Fails when: the failure is one nobody predicted

Observability

Questions you invent at 2am

- You emit enough detail to slice it afterwards
- New question → new query, not a new deploy
- Great for: why is it slow only for Nepali users on Android
- Fails when: you did not emit the detail in the first place

→ The honest version

Nobody has “observability” or “monitoring”. You have a pile of telemetry, and the question is how many unanticipated questions it can answer before you have to ship code to find out. That number is what the next three days move.

i Rule of thumb: if answering the question requires a code change and a deploy, you were monitoring, not observing.

Every incident asks the same three questions, in the same order

1 METRICS

Is something broken?

and how badly, and for whom

A number crossed a line. Cheap to store, fast to query, always on. This is what wakes you up.

2 TRACES

Where is it broken?

which service, which hop

One request's journey through every service, with timings. This is what narrows 9 suspects to 1.

3 LOGS

Why is it broken?

what exactly happened

The stack trace, the bad value, the timeout message. This is what you paste into the postmortem.



You will meet them in the reverse order in the lab — logs first, because logs are the only signal you already half-know how to read.

A team that has only metrics knows it is broken and cannot say why. A team that has only logs can explain a failure it has not yet noticed. You need all three, and you need them joined together — which is the last section of this deck.

Same event, three shapes of data

● Logs

A timestamped record of one discrete event

SHAPE

Text or JSON, one line per event

ANSWERS

What exactly happened, in detail

COST

High — grows with traffic, linearly

WEAKNESS

Volume. Finding the one line that matters.

● Metrics

A number, sampled on a schedule, over time

SHAPE

name{labels} → value @ timestamp

ANSWERS

How much, how many, how fast — in aggregate

COST

Low — grows with cardinality, not traffic

WEAKNESS

Aggregated. No individual request survives.

● Traces

The causal path of one request across services

SHAPE

A tree of spans sharing a trace_id

ANSWERS

Where the time went, and in what order

COST

Very high at 100% — hence sampling

WEAKNESS

Needs propagation everywhere, or it breaks.

★ The trade is always the same: detail per event versus cost per event. Logs keep everything and cost the most. Metrics throw away everything except a number, and cost almost nothing.

One outage, seen three ways

Payment starts failing at 14:22. Here is what each signal shows you — and what it refuses to tell you.



METRICS

SAW

Error ratio on /orders went from 0.2% to 27% at 14:22:15.

COULD NOT SAY

Which of the 40,000 failed orders, and why — no idea.

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
```



TRACES

SAW

Every failing request spends 2,000ms in payment → db, then times out.

COULD NOT SAY

Tells you where the time went. Not what the database said.

```
trace 7f3a2c... : gateway 2043ms → payment 2011ms → db  
2000ms ✗
```



LOGS

SAW

connection pool exhausted, max=10, waiting=193 — in the payment service.

COULD NOT SAY

Was there in every incident. Also in the 12 that were fine.

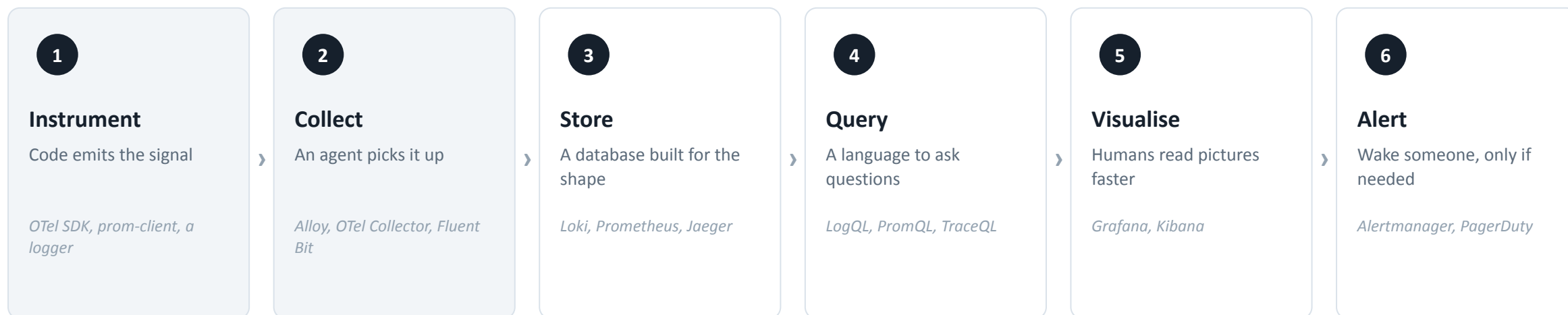
```
{service="payment"} |= "pool" | json | level="error"
```



Notice the handoff: the metric gave you a time, the trace gave you a service, the log gave you the cause. Any one alone is a longer night.

The pipeline is always the same six steps

Datadog, Grafana, CloudWatch, a shell script — all of them are this. Only the boxes change.



✓ Steps 1 and 2 are the ones you own

Everything downstream can be swapped. Your instrumentation cannot — it is in your source code. This is the whole argument for OpenTelemetry, and we come back to it at the end.

\$ Steps 3 to 6 are what you pay for

“Buy an observability platform” means buying steps 3–6 as a service. “Self-host the LGTM stack” means running them yourself. The signals are identical either way.

One application, instrumented in front of you

Chiya Shop — a tea ordering service. Small enough to read, broken enough to be interesting.

THE APPLICATION

loadgen

constant traffic

›

nginx

reverse proxy

›

api × 2

the code we edit

›

postgres

orders

›

redis

catalog cache



DAY 1

Logs

Grafana Alloy

Loki

Grafana

The app prints to stdout. Alloy tails the container, Loki stores, Grafana queries.



DAY 2

Metrics

prom-client

Prometheus

Grafana

The app exposes /metrics. Prometheus scrapes it every 15s and keeps the numbers.



DAY 3

Traces

OpenTelemetry SDK

Jaeger

The app emits spans over OTLP. Jaeger stitches them into one request tree.



Nothing here persists and nothing here is secured. It is a teaching rig, not a starter template. We will say why at the end of each day.

DAY 1

Logs

The signal you already produce by accident. Every `console.log` you have ever written was a log line — just one nobody could search.

Grafana Alloy

Loki

Grafana

LogQL

01

A log is one timestamped record of one discrete event

```
2026-07-29T14:22:07.443Z ERROR payment order_id=8821 user=u_301 attempt=2
payment gateway timed out after 2000ms (pool exhausted: max=10 waiting=193)
```

When

ISO-8601, UTC, milliseconds. Never local time — you will be correlating across machines.

How bad

A level: DEBUG / INFO / WARN / ERROR. This is what lets you filter 99% of the noise away.

Who

The service, instance and version. Without this, three containers' output is one soup.

What

The event, plus every identifier you will want to search by later.



A log line is for a human, eventually

Someone will read it at 2am with no context. Write it so the sentence stands alone: what failed, what the values were, and what the code was trying to do.



Which is why “error” is not a log line

“Something went wrong”, “here”, “done” and a bare stack trace with no identifiers are all real lines from real production codebases. They cost storage and return nothing.



Test for a good log line: could a colleague who has never seen this code act on it, without opening the source?

Structured logs, or you are writing regexes forever

UNSTRUCTURED — written for a human, parsed by nobody

```
console.log(`order ${id} failed for ${user}` +  
  ` after ${ms}ms`);  
  
order 8821 failed for u_301 after 2043ms
```

- To count failures per user you must write a regex
- Someone reorders the sentence → every regex breaks
- Multi-line stack traces arrive as 30 separate events

STRUCTURED — written for a machine, rendered for a human

```
log.error({ order_id: id, user, duration_ms: ms },  
  'order failed');  
  
{ "level": "error", "order_id": 8821, "user": "u_301",  
  "duration_ms": 2043, "msg": "order failed" }
```

- Filter, group and count by field, with no parsing
- Add a field tomorrow — nothing that exists breaks
- One event is one JSON object, however many lines it prints

→ The rule: log to stdout, as JSON, and let something else move it

Your application should not know where its logs end up. No file paths, no rotation, no shipping code, no vendor SDK in the logger. Write JSON to stdout; the platform captures it. This is the twelve-factor rule and it is the reason you can change log vendors in an afternoon.

! Cost of the mistake: unstructured logs are not “slightly worse”. They are unqueryable at volume, and re-logging means re-deploying every service.

Levels are a budget. Fields are a liability.

USE THE LEVELS AS THEY WERE MEANT

ERROR

A human must eventually act. Page-worthy or ticket-worthy.

WARN

Degraded but handled. A retry succeeded; a fallback fired.

INFO

A business event happened. Order placed. User registered.

DEBUG

Developer detail. Off in production, on for one service, briefly.

If everything is ERROR, nothing is. Levels work only if review enforces them.

NEVER LOG THESE. NOT ONCE. NOT AT DEBUG.

- Passwords, tokens, API keys, session cookies, card numbers
- Full request or response bodies “just while we debug”
- Personal data you would not put in an email: names, phone numbers, addresses, national ID
- Anything a regulator would call personal data — logs are copied to five systems and kept for a year

A leak in logs is a breach. Redact at the logger, not at the query.

i Log the identifier, not the value

`user_id=u_301` is searchable, joinable and harmless. `email=ram@example.com` is a liability that lives in your log store, your backups, and your vendor's backups. The identifier gets you to the same customer through a system that is actually access-controlled.

★ In the lab: every log line is public to the whole class. Treat it as production. That is the point.

Three services, interleaved, and no way to tell which lines belong together

```
14:22:07.441 gateway POST /orders received
14:22:07.442 gateway POST /orders received
14:22:07.443 catalog lookup tea_id=44
14:22:07.449 payment charging 450 NPR
14:22:07.451 catalog lookup tea_id=91
14:22:09.443 payment timeout after 2000ms
14:22:09.444 gateway 500
14:22:09.512 gateway 201
```

which request failed? both? one? which tea?

```
14:22:07.441 gateway req=alf POST /orders
14:22:07.442 gateway req=b7c POST /orders
14:22:07.443 catalog req=alf lookup tea_id=44
14:22:07.449 payment req=alf charging 450 NPR
14:22:07.451 catalog req=b7c lookup tea_id=91
14:22:09.443 payment req=alf timeout 2000ms
14:22:09.444 gateway req=alf 500
14:22:09.512 gateway req=b7c 201
```

one field. now the story reads itself.

1 Generate it at the edge

The first service to touch the request mints an ID — or accepts one from the load balancer. Never generate it twice.

2 Pass it on every hop

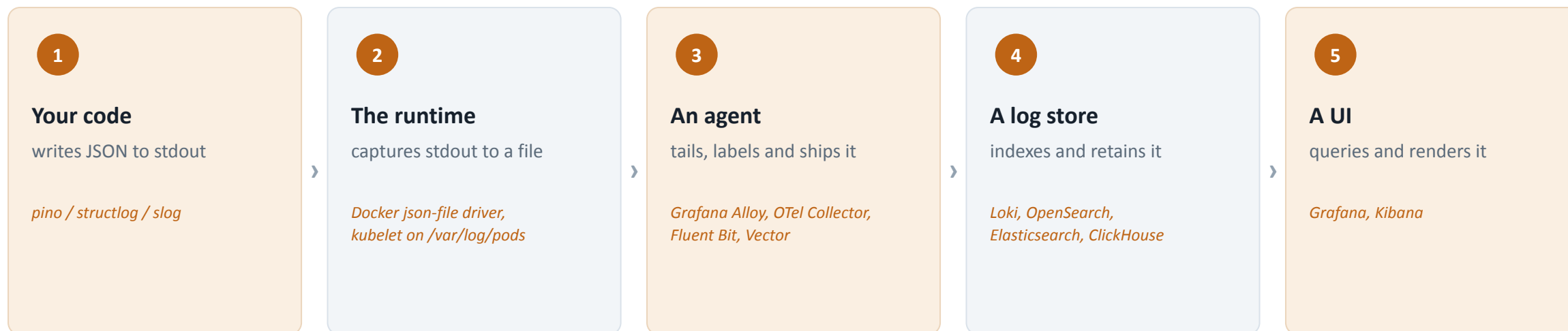
An HTTP header on outbound calls, a field on every queue message. Miss one hop and the chain is cut from there down.

3 This is a trace, badly

You have just reinvented distributed tracing without the timings or the parent-child tree. On Day 3 we use `trace_id` instead — and get all of it for free.

→ So: put the `trace_id` in your log lines from the start. One field turns your logs and your traces into one searchable thing — the payoff slide at the end of this deck.

From a print statement to a query box



3 Why the agent is a separate process

It survives your app crashing, buffers when the backend is down, adds host and container metadata your app cannot know, and lets you change backends without touching code.

! Promtail is dead — it reached end of life on 2 March 2026

Every tutorial older than a year still uses it. Grafana Alloy replaces it and is a distribution of the OpenTelemetry Collector, so it handles logs, metrics and traces in one agent.

! The agent needs the Docker socket or the host log directory to do its job. That is root-equivalent access on the host. Treat your log agent's configuration as security-sensitive infrastructure, not as a sidecar nobody reviews.

Index everything, or index almost nothing

This single design choice explains most of the price difference between log products.

Full-text index

Elasticsearch, OpenSearch, Splunk

- Every word in every line goes into an inverted index
- Searching arbitrary text is fast, at any age of data
- The index can be as large as the data itself
- You pay in RAM, disk and cluster operations — forever

Index the labels only

Loki (and Loki-shaped stores)

- Only a small set of labels is indexed; the line itself is compressed and stored whole
- Cheap to store, cheap to run, object storage friendly
- A text search is a brute-force scan of the matching streams
- You pay in query time — so the label selector has to narrow it first

```
{service="payment", env="prod"} |= "pool exhausted"
  L indexed: picks the streams           L not indexed: scanned inside them
```



The consequence you will feel in the lab: in Loki a query with no label selector, or one that matches everything, is not slow — it is refused or it times out. “Search all logs for this word” is a Splunk habit that does not transfer. Narrow by labels first, always.

LogQL: pick a stream, filter it, parse it, count it

1 Select the stream

1 Label matchers only. This is the indexed part, and it is mandatory.

```
{service="payment", env="prod"}
```

2 Filter the line

2 Substring match on the raw line. |= contains, != excludes, |~ regex.

```
{service="payment"} |= "timeout" != "healthcheck"
```

3 Parse into fields

3 Now the JSON you logged pays off: filter on a field, not on text.

```
{service="payment"} | json | duration_ms > 1000
```

4 Turn logs into a number

4 A log query becomes a metric. This is how you alert on logs — and it is Day 2's idea, early.

```
sum by (service) (rate({env="prod"} | json | level="error" [5m]))
```

★ Every log query language is these four rungs in a different syntax. Splunk SPL, Elasticsearch KQL, CloudWatch Logs Insights, Azure KQL — learn the shape once.

Logs are the line item that surprises everyone

A worked example. Small service, nothing exotic, no mistakes yet.

500

requests per second

a modest service

×

8

log lines per request

in, out, and six in between

×

~350 GB

of logs per day

at ~1 KB per line

×

× 30 days

of retention

and now it is ~10 TB

THE FOUR LEVERS, IN ORDER OF LEVERAGE

- Retention — 30 days of everything is a choice, not a law. Most questions are asked within 48 hours.
- Volume — delete the DEBUG line in the hot loop. One line can be half your bill.
- Sampling — keep 1 in 100 successful requests, keep 100% of errors.
- Tiering — hot for a week, cheap object storage after that.

HOW THE BILL ACTUALLY EXPLODES

- Someone turns on DEBUG in production to chase a bug, and leaves it on
- A retry loop starts logging on every attempt during an outage — the bill spikes exactly when you are already having a bad day
- A health check endpoint logs, and it is polled every second, per replica, forever



Every SaaS log vendor prices on ingested volume. That means your logging habits, written months ago by someone who has left, set the invoice.

Where you can put logs

Four buying models. The signal is identical in all four — what changes is who runs it and how you pay.

● Run it yourself

Open source, your servers

Grafana Loki

OpenSearch

Elasticsearch + Kibana

Graylog

VictoriaLogs

Quickwit

ClickHouse

SigNoz

OpenObserve

Cheapest licence, most expensive engineer. You are now on call for the thing that tells you when you are on call.

● Managed open source

Same tools, someone else's problem

Grafana Cloud Logs

Elastic Cloud

AWS OpenSearch Service

Logz.io

Aiven

Same query language and dashboards, no cluster to operate. The usual first move once self-hosting hurts.

● Full-platform SaaS

Logs bundled with everything else

Datadog

Splunk (Cisco)

New Relic

Dynatrace

Sumo Logic

Coralogix

Better Stack

Axiom

Mezmo

Fastest to value, deepest correlation, and the bill that gets escalated to finance. Priced per GB ingested.

● Your cloud provider

Already there, already billing

AWS CloudWatch Logs

Azure Monitor Logs

Google Cloud Logging

Zero setup for cloud infrastructure logs, weakest query experience of the four, and the default nobody chose.



We use Loki because it runs on a laptop and shows the ideas clearly — not because it is the right answer at your future job.

DAY 2

Metrics

Numbers over time, deliberately stripped of detail so they stay cheap enough to keep forever and query in milliseconds.

Prometheus

PromQL

Grafana

prom-client



02

A metric is a number, with labels, sampled on a schedule

```
http_requests_total{service="payment", route="/orders", method="POST", status="500"} 4821
```

└─ metric name └─ labels: the dimensions you can slice by └─ value

It is a time series

One unique combination of name and labels is one series. Prometheus stores a value for it every scrape — typically every 15 seconds, forever.

It is already aggregated

4,821 requests failed. Which ones? Gone. You traded the individual events away in exchange for a number you can keep cheaply.

It is dimensional

Labels are what make one metric answer many questions: by service, by route, by status, by version — without emitting a new metric for each.



This is why metrics can wake you up

Cheap to store, fast to query, complete by default — no sampling. An alert can evaluate every 15 seconds against a year of history without breaking anything.



And why metrics can never explain anything

“27% of orders failed” is a metric. “Order 8821 failed because the connection pool was exhausted” is not, and never will be. Metrics are the alarm, not the answer.



Cost scales with the number of unique label combinations — not with traffic. Ten requests and ten million cost the same.

Four types. Using the wrong one fails silently.

Counter

Only ever goes up. Resets to zero when the process restarts.

EXAMPLE `orders_total, errors_total, bytes_sent_total`

QUERY IT WITH `rate(x[5m])` — **never the raw value**

The raw number is meaningless (“4,821 since when?”). The rate of change is the useful thing.

Gauge

Goes up and down. A measurement of right now.

EXAMPLE `queue_depth, temperature, tea_stock, memory_bytes`

QUERY IT WITH **the raw value, or `delta()` / `deriv()`**

Sampled every 15s, so anything that happens between scrapes is invisible. Spikes hide here.

Histogram

Counts observations into buckets, plus a sum and a count.

EXAMPLE `http_request_duration_seconds`

QUERY IT WITH `histogram_quantile(0.95, ...)`

How you get p95 latency. Percentiles are computed at query time from bucket counts.

Summary

Like a histogram, but quantiles are computed in the client.

EXAMPLE `rarely what you want in new code`

QUERY IT WITH **the pre-computed quantile**

You cannot aggregate a summary's quantiles across instances. Averaging two p95s is not a p95.



`rate()` on a gauge returns a number. It is a wrong number, and Prometheus will not warn you — every decrease is read as a process restart. You will do this deliberately in the lab so you recognise the shape of it.

Pull or push — and why Prometheus chose pull

PULL (SCRAPE)

Prometheus asks each target for its numbers

```
GET http://api:3000/metrics    every 15s  
→ a plain text page of name{labels} value
```

- The scrape itself tells you the target is alive (up = 0 or 1)
- You can open /metrics in a browser and read it — debugging is trivial
- Prometheus controls the rate, so a busy app cannot flood it
- Needs network reachability and service discovery to find targets

PUSH

The application sends its numbers onward

```
app → StatsD / OTLP / vendor agent → backend  
→ whenever the app decides to send
```

- Works when the target is unreachable: serverless, batch jobs, browsers, IoT
- No service discovery needed — the sender knows where to send
- Silence is ambiguous: is it healthy and idle, or dead?
- The default in OpenTelemetry, and in most commercial agents

→ In practice you will run both, and that is fine

Long-lived services get scraped. Short-lived jobs push to a Pushgateway, or an OTel Collector receives pushes and exposes a scrape endpoint on their behalf. The collector translating between the two models is one of the most common patterns in production today.

- ★ A cron job that runs for 4 seconds cannot be scraped every 15. That single fact is why Pushgateway exists — and why it should not be used for anything else.

Cardinality: every unique label combination is a new series

```
const route = req.route.path;      // '/teas/:id'   → one series, forever
const route = req.originalUrl;     // '/teas/8821' → one series per tea. per method. per status.
```

THE ARITHMETIC OF ONE BAD LABEL

1 series in the request counter

13 in the histogram — 11 buckets, plus `_sum` and `_count`

× method × status because labels multiply, they do not add

≈ 14+ new series for every single unique URL
Random URLs from a script become a denial-of-service on your monitoring.

NEVER PUT THESE IN A LABEL

user ID

order ID

request ID

trace ID

email

session ID

raw URL

IP address

error message

timestamp

The test: can you write down every possible value on one line? If not, it is not a label — it belongs in a log line or a span, which are built for exactly this.

! Damage from a cardinality mistake outlives the fix

Revert the code and the bad series do not disappear — they sit in memory and on disk until the retention window expires. On a real cluster with 30-day retention, one afternoon's mistake is a month of degraded queries.

! “It is only one more label” is the most expensive sentence in this module. You will detonate this yourself in the lab and watch the series count climb.

PromQL: five queries, and two silent failures

1 `http_requests_total`

The raw counter. One value per series, right now. Almost never what you want.

2 `rate(http_requests_total[5m])`

Per-second rate over a 5-minute window, correct across counter resets.

3 `sum by (service) (rate(http_requests_total[5m]))`

Requests per second per service. rate first, then sum — always in that order.

4 `sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
/ sum by (service) (rate(http_requests_total[5m]))`

The error ratio, your most important SLI. Note 5.. and not 4.. — a 404 is the client being wrong.

5 `histogram_quantile(0.95, sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))`

p95 latency per route, computed from bucket counts at query time.

1 **`sum(rate(...)) not rate(sum(...))`**

Summing counters first, then rating, turns one restart into a cliff in your graph and a false alert.

2 **`le` must survive the `by` clause**

Drop `le` from a `histogram_quantile` and you get NaN. No error, no warning. The single most common PromQL mistake.

Three frameworks, so you stop guessing

They overlap heavily, on purpose. Pick one, apply it to every service, and stop debating.

RED

For every request-driven service

- Rate — requests per second
- Errors — failed requests per second
- Duration — the latency distribution

Three metrics per service, and you can build the same dashboard for all of them. Start here.

USE

For every resource: CPU, disk, pool, queue

- Utilisation — how busy it is
- Saturation — how much work is queued and waiting
- Errors — how often it failed outright

Saturation is the one people forget, and it is the one that predicts the outage.

Four Golden Signals

Google SRE's version. RED plus saturation.

- Latency — split fast successes from fast failures
- Traffic — demand on the system
- Errors — explicit, implicit, and by policy
- Saturation — how full the tightest resource is

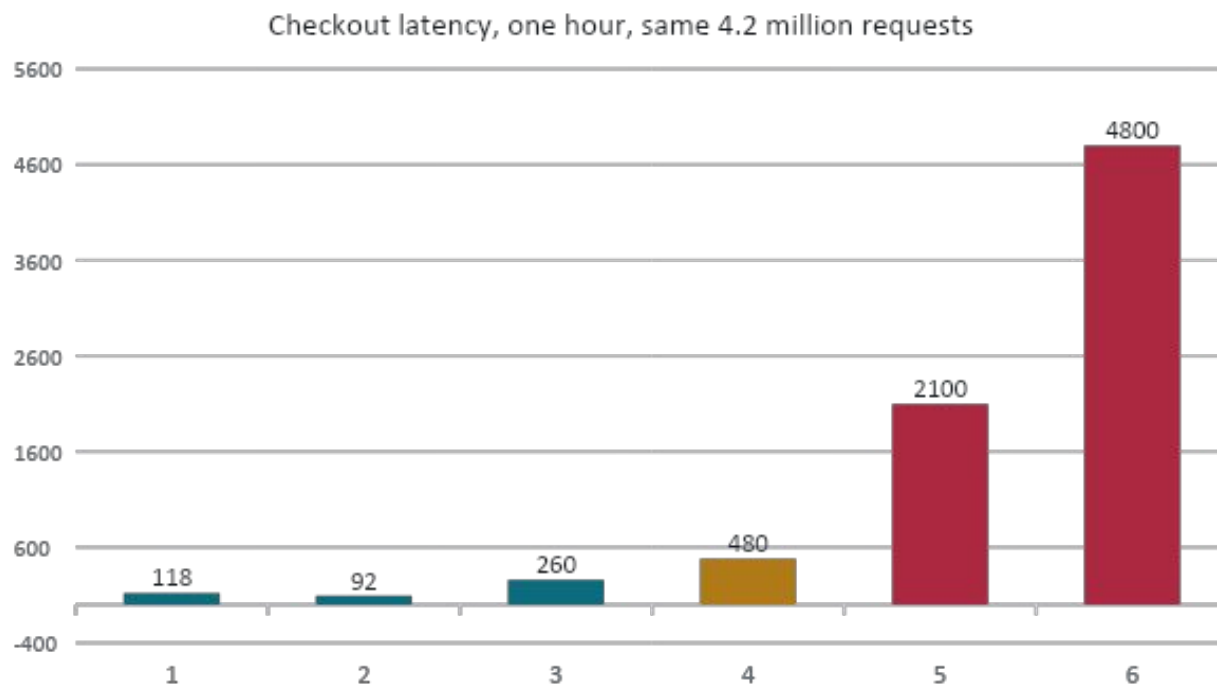
The most widely quoted framework. If someone asks in an interview, this is the answer they want.



And then measure the business, not just the infrastructure

orders_placed_total tells you more in one line than a wall of CPU graphs. When orders drop to zero, something is wrong — even if every technical metric is green, which is exactly the failure mode that infrastructure dashboards miss. Ask: what number would the person who pays for this system look at first?

The average is the number that hides the outage



! “Average latency is 118ms”

True, and useless. It is one number describing a distribution with two humps in it: fast cache hits, and slow requests waiting on a pool. Nobody experiences 118ms.

✓ “1 in 100 users waits 2.1 seconds”

Same data, actionable. At 4.2 million requests that is 42,000 people, and the ones hitting p99 are disproportionately your heaviest, most valuable users — more data, more work, slower path.

THREE RULES THAT FOLLOW

- Alert on percentiles, never on averages — an average can stay flat through a total outage of 5% of traffic.
- You cannot average percentiles across instances. p95 of A and p95 of B is not p95 of A+B. Aggregate the buckets, then compute.
- Pick the percentile from the promise you made. A p99 target means one in a hundred is allowed to be bad — decide that on purpose.

Where you can put metrics

Same four buying models as logs. Note how many of these speak PromQL — that is not an accident, it is the moat Prometheus built.

Run it yourself

Open source, your servers

Prometheus

VictoriaMetrics

Grafana Mimir

Thanos

Cortex

InfluxDB

Graphite

OpenTSDB

Prometheus on one box is easy. Prometheus that is highly available and keeps two years of data is a project — that is what Mimir, Thanos and VictoriaMetrics are for.

Managed open source

Prometheus-compatible, hosted

Grafana Cloud Metrics

Amazon Managed Prometheus

Azure Monitor managed Prometheus

Google Managed Prometheus

VictoriaMetrics Cloud

Your dashboards, alert rules and PromQL all move across unchanged. The lowest-regret migration in this entire deck.

Full-platform SaaS

Metrics inside a bigger product

Datadog

New Relic

Dynatrace

Splunk Observability (Cisco)

Chronosphere (Palo Alto)

Sumo Logic

Dash0

Last9

Usually billed per host and per custom metric, which is where cardinality stops being an engineering concern and becomes a finance one.

Your cloud provider

Already collecting, already charging

AWS CloudWatch Metrics

Azure Monitor Metrics

Google Cloud Monitoring

Unbeatable for the cloud's own resources — load balancers, queues, managed databases. Awkward and expensive for your application's own metrics.



The transferable skill is PromQL and the metric type model. Almost every option above either speaks PromQL or has copied its data model.

DAY 3

Traces

One request, followed across every service it touched, with the timing of each hop and the causal order preserved.

OpenTelemetry

OTLP

Jaeger

Context propagation

03

“p95 is 3 seconds.” Which of the nine hops?

Metrics told you the total. Logs told you each service's opinion of itself. Neither adds up to a request.

Every service says it is fast

Each one measures only its own work, not the time it spent waiting for someone else. All nine dashboards are green and the user waits three seconds.

Timestamps do not add up

Clocks drift between machines. Ordering log lines from three hosts by timestamp gives you a plausible story that is wrong.

Fan-out is invisible

One request became forty calls. The metric shows forty fast calls. It cannot show that they were all for one user, in series.

WHAT A TRACE ADDS

Causality and duration, for one request, across every process

Order	What called what, in what sequence — from the code, not from clocks.	Duration	How long each hop took, and how much of it was waiting on a child.
Structure	Serial or parallel, one call or forty, retried or not.	Attachment	Errors, SQL statements, status codes and identifiers, attached to the exact hop that produced them.

 A trace is a stack trace that crosses process boundaries. That one sentence is the whole idea.

A trace is a tree of spans sharing one trace_id



trace_id

One 16-byte ID, generated once at the edge, carried by every span in the request. This is the join key for everything.

span

One unit of work: an HTTP handler, a query, a function. It has a name, a start, a duration, a status, and attributes.

parent_span_id

What makes it a tree rather than a list. This is where the structure and the ordering come from.

attributes & events

Key-value detail on a span: http.status_code, db.statement, user id, exception. High cardinality is fine here — unlike metrics.



Read the picture: catalog was fast, payment took 1811ms, and almost all of it was one child span waiting on a database connection. You have your answer in five seconds.

Propagation: one header, and everything depends on it

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
      L version  L trace-id (16 bytes)          L parent span-id  L sampled flag
```

HOW IT MOVES

- 1 The first service creates a `trace_id` and a root span
- 2 On every outbound call it injects `traceparent` into the headers
- 3 The receiving service extracts it, and starts a child span
- 4 Repeat, all the way down. One ID, many processes, one tree.

WHERE THE CHAIN BREAKS

- A queue — nothing carries the header unless you put it in the message yourself
- A thread pool or a background job that outlives the request
- One service in the chain not instrumented at all
- A proxy or gateway configured to strip unknown headers

✓ W3C `traceparent` is a standard, and that matters

Any two instrumented services interoperate, even across languages, frameworks and vendors. There is no proprietary header to agree on.

i Broken propagation is the number one tracing bug

The symptom is always the same: lots of short, orphaned traces that all start at one service. Look for the hop that dropped the header.

- ★ Instrumentation is per service, but tracing is only useful if it is everywhere. One un-instrumented hop cuts the tree at that point.

Three levels of instrumentation, and you want all three

1 Zero-code / auto

Effort: Minutes

An agent or a startup flag patches the libraries you already use. HTTP servers, HTTP clients, database drivers and queue clients all start emitting spans without you touching a line.

```
node --require
@opentelemetry/auto-instrumentations-node/register
app.js
```

Gets you 80% of the value on day one. Start here, always.

2 Library / framework

Effort: Hours

You add the SDK and configure it properly: service name, version, environment, resource attributes, sampler and exporter. This is what makes the traces navigable rather than merely present.

```
service.name=payment service.version=1.4.2
deployment.environment=prod
```

Without service.name every span says “unknown_service”. Do this on day one too.

3 Manual spans

Effort: Ongoing

You wrap the parts that matter to your business and add attributes only you know: which pricing rule fired, which cache tier answered, which retry attempt this is.

```
span.setAttribute('order.value_npr', 450)
```

This is where tracing goes from useful to indispensable. Add it where you have been burned.

★ The order matters. Teams that start at level 3 write beautiful manual spans for one service and never finish the other eight, so the traces are still cut in half. Turn auto-instrumentation on everywhere first, then go deep where it hurts.

Sampling: you cannot keep every trace, and you do not need to

At 500 requests/second with 9 spans each, 100% sampling is roughly 390 million spans per day — from one service.

Head sampling

Decided at the start, at the root span

✓ GOOD

- Trivial to run — no buffering, no extra infrastructure
- The decision rides along in the traceparent flag, so the trace is whole or absent
- Predictable cost: 1% means 1%

✗ COSTS

- You throw away the interesting traces at random
- The one 8-second request is probably in the 99% you dropped

Tail sampling

Decided at the end, once the trace is complete

✓ GOOD

- Keep 100% of errors, 100% of slow traces, 1% of boring ones
- You keep exactly the traces you would want to look at

✗ COSTS

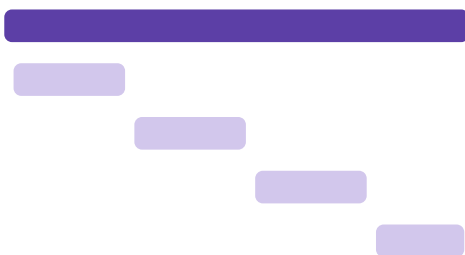
- The collector buffers every span until the trace finishes — memory, and a delay
- All spans of one trace must reach the same collector, so it needs routing

 The trap: once you sample, you can no longer count anything from traces. “3 errors in my sampled traces” at 1% sampling means roughly 300 errors, or possibly 50 — you do not know. Count with metrics. Investigate with traces. Never swap those jobs.

Four shapes, and what each one means

You will not read span names in an incident. You will read shapes, and the shape tells you the bug.

The staircase

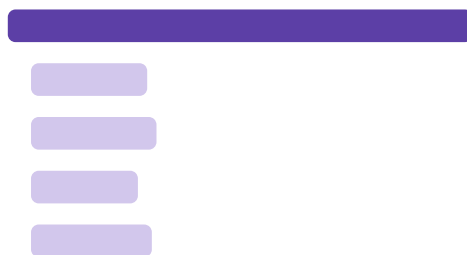


Serial calls in a loop. Classic N+1: one query per item, forty times.

FIX

Batch it, or fetch in parallel.

The comb



Parallel fan-out. Healthy — total time is the slowest child, not the sum.

FIX

Nothing, unless one tooth is long.

The gap



Unaccounted time inside the parent: queueing, GC, connection pool wait, or an un-instrumented call.

FIX

Add a span there. The gap is the bug.

The long tail



One child dominates everything. This is the good case — your suspect is already identified.

FIX

Go read that service's logs.



The gap is the most valuable of the four and the easiest to miss, because nothing is drawn there. Time inside a parent that no child accounts for is time your instrumentation cannot see — which is usually where the problem is hiding.

Where you can put traces

This column used to be called APM, and it was the most locked-in part of the stack. OpenTelemetry broke that, recently and deliberately.

● Run it yourself

Open source, your servers

Jaeger

Grafana Tempo

Zipkin

SigNoz

Uptrace

HyperDX

OpenObserve

Apache SkyWalking

Jaeger for learning and for modest volume. Tempo when you want cheap object-storage retention and Grafana-native correlation.

● Managed open source

OTLP in, hosted backend

Grafana Cloud Traces

AWS X-Ray (via ADOT)

Azure Application Insights

Google Cloud Trace

You keep OTel instrumentation and hand over the storage. Switching between these is a change to one exporter endpoint.

● Full-platform APM

Traces plus everything, correlated

Datadog APM

New Relic

Dynatrace

Honeycomb

Splunk APM

IBM Instana

Elastic APM

Sentry

Middleware

Dash0

The deepest analysis, especially on high-cardinality trace data. Honeycomb in particular is built around exactly this question.

● Cautionary tale

Read this column carefully

ServiceNow Cloud Observability

formerly Lightstep

Built by the people who created distributed tracing at Google and co-founded OpenTelemetry. Acquired in 2021, and shut down on 1 March 2026 with no successor product and no migration path. Customers on OTel changed one endpoint. Customers on the proprietary agent re-instrumented everything.



That last column is the single best argument in this deck for keeping your instrumentation vendor-neutral. Which is the next section.

TYING IT TOGETHER

One standard, one agent, one story

Three signals are only worth three days if they join up. This is the part that decides whether you have a stack or three separate tabs.

OpenTelemetry

OTLP

Collector

Correlation

Build vs buy

04

OpenTelemetry: instrument once, send anywhere

The second most active project in the CNCF after Kubernetes, and the reason the vendor slides you just saw are less frightening than they used to be.

BEFORE

The vendor's agent lived in your source code

- A different SDK per vendor, per language
- Changing vendor meant re-instrumenting every service
- Evaluating a second vendor meant running two agents
- The switching cost was the product strategy

NOW

One API, one SDK, one wire protocol

- Instrument with OTel, export over OTLP
- Changing backend is an endpoint and a credential
- Send the same data to two backends at once, to compare
- Every serious vendor accepts OTLP, because customers demand it

WHAT OPENTELEMETRY ACTUALLY GIVES YOU

API + SDKs

Per language. Create spans, record metrics, emit logs.

OTLP

The wire protocol every backend now speaks.

Semantic conventions

Agreed names: `http.request.method`, `db.system`. This is why dashboards are portable.

The Collector

A vendor-neutral pipeline that receives, processes and exports.



Maturity differs by signal: tracing is stable and boring in the good sense, metrics are stable, logs are the newest and still catching up in some languages. Check the status page for your language before you promise anything.

The Collector: where you fix things without redeploying code

Receivers

OTLP, Prometheus scrape, Jaeger, Zipkin, filelog, StatsD, Kafka

Processors

batch, memory limiter, sampling, filtering, redaction, adding resource attributes

Exporters

Loki, Prometheus, Tempo, Jaeger, Datadog, any OTLP endpoint — several at once

WHAT THIS BUYS YOU IN PRACTICE

- Drop the metric that is 80% of your bill — config change, no deploy, no code review
- Redact a field somebody logged by mistake, at the edge, before it leaves your network
- Add environment and region to every signal from every service, in one place
- Switch backends, or fan out to two, without touching an application
- Absorb a backend outage in a buffer instead of losing the telemetry

WHERE IT RUNS

Agent

One per host or as a sidecar. Close to the app, adds host metadata, minimal buffering.

Gateway

A central cluster. Where tail sampling, heavy processing and cost control actually live.

Both

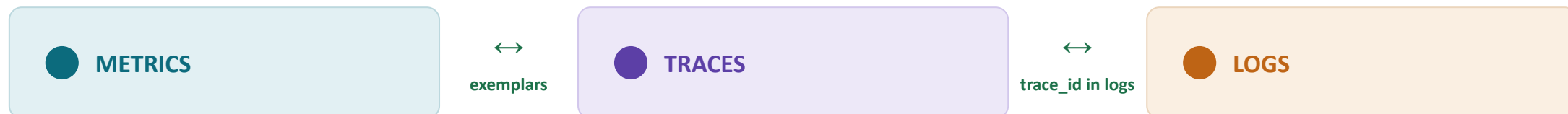
The normal production shape: agents collect and forward, a gateway decides and exports.

Grafana Alloy, which we use on Day 1, is a distribution of this Collector.

★ The reason this matters: everything you would otherwise have to beg eight teams to change in their code, you can change in one YAML file that you own.

Three signals are worth more joined than separate

● All three



THE WORKFLOW THIS ENABLES — THREE CLICKS, NOT THREE TABS

- 1 An alert fires on the p99 latency panel
- 2 Click an exemplar dot on the graph → the exact slow trace opens
- 3 Find the slow span → click through to that span's log lines
- 4 Read the error. Elapsed: about ninety seconds.

WHAT IT COSTS YOU TO GET THIS

- Put `trace_id` into every log line — usually one logger hook
- Turn on exemplars in your histogram client and your backend
- Use the same `service.name` string in all three signals
- That is genuinely most of it

WHAT BREAKS IT

- The service is called `payment` in metrics and `payment-svc` in traces
- Logs go to one vendor and traces to another, with no shared link
- Sampling drops the trace the exemplar points at, so the link dead-ends
- Nobody agreed on names, so every join is manual

- ★ Naming discipline is not bureaucracy. It is the difference between one investigation and three, every single time.

Who does what, and how you pay for it

	Run it yourself	Managed open source	Full-platform SaaS	Your cloud provider
Logs	Loki, OpenSearch, Elasticsearch, Graylog, VictoriaLogs, ClickHouse	Grafana Cloud, Elastic Cloud, AWS OpenSearch Service, Logz.io	Datadog, Splunk (Cisco), New Relic, Sumo Logic, Coralogix, Better Stack, Axiom	CloudWatch Logs, Azure Monitor Logs, Google Cloud Logging
Metrics	Prometheus, VictoriaMetrics, Mimir, Thanos, Cortex, InfluxDB, Graphite	Grafana Cloud, Amazon / Azure / Google managed Prometheus	Datadog, New Relic, Dynatrace, Splunk Observability, Chronosphere (Palo Alto)	CloudWatch Metrics, Azure Monitor Metrics, Google Cloud Monitoring
Traces	Jaeger, Tempo, Zipkin, SigNoz, Uptrace, SkyWalking	Grafana Cloud Traces, AWS X-Ray, Azure App Insights, Google Cloud Trace	Datadog APM, Honeycomb, Dynatrace, Instana, Elastic APM, Sentry, Middleware, Dash0	X-Ray, Application Insights, Cloud Trace
Collection	OTel Collector, Grafana Alloy, Fluent Bit, Vector, Telegraf	Vendor-run OTLP endpoints, AWS Distro for OTel	Each vendor's own agent (and, now, OTLP)	CloudWatch Agent, Azure Monitor Agent, Ops Agent

What actually differs between columns:

who is on call for the observability stack itself, how the bill is metered (GB ingested, hosts, series, users), and how hard it is to leave. The signals, the query concepts and the instrumentation are the same in every column — which is exactly why learning the concepts beats learning a product.

Self-host or pay someone

Self-host the open-source stack

✓ CHOOSE IT WHEN

- You have a platform team, or you are trying to build one
- Data cannot leave your network for legal or regulatory reasons
- Telemetry volume is large enough that SaaS pricing is genuinely painful
- You want the skills in-house, on purpose

✗ WHAT IT COSTS YOU

- Nights and weekends now include your monitoring stack
- Storage, retention, upgrades, scaling and HA are all yours
- The classic failure: the observability stack dies during the outage it was meant to explain

Buy a managed platform

✓ CHOOSE IT WHEN

- Small team, and every engineer-hour has an obvious better use
- You need correlation and dashboards working this month, not this quarter
- You want somebody to call at 3am who is not you

✗ WHAT IT COSTS YOU

- The bill scales with your success and it is rarely linear
- Per-host, per-GB and per-custom-metric pricing makes engineering choices into finance choices
- Leaving is expensive unless you kept instrumentation vendor-neutral



The answer for almost everyone: buy the backend, own the instrumentation. Emit OpenTelemetry, run your own collector, and let the expensive part be replaceable. Then the vendor decision stops being permanent, and you can make it on price.

What the next three days actually look like

DAY 1 Logs

- Debug a failure with nothing but docker logs, and fail
- Add structured JSON logging, then labels
- Ship with Alloy, store in Loki, query with LogQL
- Break it: an unbounded label, and a log line that leaks

DAY 2 Metrics

- Expose /metrics and read what the client already gives you
- Counters, gauges and histograms on the same request path
- Build a RED dashboard from scratch in PromQL
- Break it: rate() a gauge, then detonate the cardinality

DAY 3 Traces

- Turn on OTEL auto-instrumentation and watch traces appear
- Read waterfalls in Jaeger and find the slow hop
- Add manual spans and attributes where they matter
- Break the propagation chain, then join logs to traces

HOW EACH DAY RUNS

Problem first

You get the broken thing before you get the tool. The tool has to earn its place.

Guided lab

Everything is provided and commented. You are reading and running, not typing YAML from memory.

Break it on purpose

Every day ends with a deliberate failure, because the failure modes are the actual lesson.

What we skipped

Each day closes with an honest list of what a real deployment needs and we left out.

Twelve rules that survive any tool change

● Log JSON to stdout

Never files, never a vendor SDK in your logger.

● Log identifiers, not values

user_id, not email. A log leak is a breach.

● Put trace_id in every log line

One field, and your signals join up for free.

● Levels are a budget

If everything is ERROR, your error alert is noise.

● Cardinality is the metric killer

If you cannot list the values, it is not a label.

● rate() before sum(), always

And keep le, or histogram_quantile returns NaN.

● Alert on percentiles, not averages

An average stays flat through an outage.

● Measure the business, not just the box

orders_total beats any CPU graph.

● Auto-instrument everywhere first

A trace is only as good as its worst hop.

● Sample traces, count with metrics

Never compute totals from sampled data.

● One service.name, used everywhere

Naming discipline is what makes correlation work.

● Own your instrumentation

Emit OTLP. Then the backend is a purchasing decision.

★ None of these mention a product. That is deliberate — products change every three years, and these do not.



You cannot debug what you cannot see.

And you cannot see what you did not decide to emit — months ago, in a pull request nobody thought was about observability.

TOMORROW

Day 1 — Logs. Bring the lab repo cloned and Docker running.

BEFORE THEN

Find out which logging library your language uses, and whether it emits JSON.

THE QUESTION TO KEEP ASKING

“If this broke at 2am, what would I wish I had emitted?”